



INTERNAL

Engineering Principles & Standards

Quenyx vOPS HUB — v1.0.0 · Document v2.0

Document Version	2.0
Software Version	v1.0.0
Applies To	Quenyx vOPS HUB v1.0.0
Classification	Internal
Owner	Engineering
Status	Released
Last Updated	2026-06-29
Document Type	Engineering standards

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0 (Documentation Pack v2.0).

Table of Contents

- 1. Platform principles
- 2. No fake data
- 3. Source-of-truth database
- 4. UUID-first
- 5. Provenance-first
- 6. Fail-closed
- 7. Deterministic before probabilistic
- 8. AI as renderer, platform as expert
- 9. Module adapter pattern
- 10. Service boundaries
- 11. Testing standards
- 12. Documentation standards
- 13. Release standards
- 14. Security-by-design
- AI Adapter Platform principles (Sprint 22)
- Automation standards (Sprint 23)
- Knowledge & collaboration standards (Sprint 24)
- Enterprise Intelligence standards (Sprint 25)

21 — Engineering Principles & Standards

Audience: The engineering org. **Purpose:** The non-negotiables that define how Quenyx is built. These are observed in the current codebase and are the bar for all future work.

1. Platform principles

- **One source of truth.** Operational + compliance data live in the platform; modules share identity, entitlements, audit, and the AI layer.
- **Thin edges, rich services.** Controllers validate and delegate; logic lives in `app/Services/**`.
- **Composable modules.** New modules plug in without re-plumbing identity/billing/AI.

2. No fake data

No `lorem`, `sample/demo/fake` controls, or placeholder business data in production paths. The corpus validator **rejects** fake-data markers. Executive metrics are **real counts**, never fabricated.

3. Source-of-truth database

The database is authoritative for business data. The AI core does **not** read business data directly — it consumes deterministic service outputs. Compliance content is imported from **official sources only**.

4. UUID-first

Domain entities (corpus, AI, RAG) use **UUIDs**. Public payloads expose UUIDs/codes — **never** raw auto-increment IDs.

5. Provenance-first

Every compliance element traces to a **source document** and an **import run**. Revisions are immutable snapshots. Nothing enters the corpus without provenance.

6. Fail-closed

Missing auth, missing entitlement, invalid input, or an uncitable AI answer → **reject**, don't degrade silently. Security and correctness defaults are restrictive.

7. Deterministic before probabilistic

A deterministic engine decides **what** is true and **what** to answer **before** any model is involved. Same inputs ⇒ same outputs. This is the core governance guarantee.

8. AI as renderer, platform as expert

The model **phrases**; it does not **decide**. Reasoning, gaps, recommendations, and citations come from deterministic engines. AI is off by default and provider-agnostic.

9. Module adapter pattern

Modules consume AI through `AiModuleAdapterInterface` (`buildContext` → `buildReasoning` → `buildPrompt`). Adapters **reuse** module services; they **never duplicate or move** business logic into the AI core.

10. Service boundaries

- Provider access only through `AiProviderRegistry` / provider classes. **No hardcoded models.**
- No direct OpenAI/HTTP calls outside provider classes (verified by static scan).
- DB access stays in the services/persistence layer, not the AI orchestration core.

11. Testing standards

- New services get deterministic unit tests; new routes get authorization/entitlement tests.
- Run `php artisan test (+ --filter=Compliance, --filter=AI)` before merge.
- Prefer deterministic fixtures over randomized data.

12. Documentation standards

- Docs live in `docs/quenyx-v1/` ; update them in the **same PR** as the code.
- Use the status legend (● / ● / ● / ●). **No fabrication.**
- Re-derive mechanical refs (Doc 08 routes, Doc 09 migrations) when they change.

13. Release standards

- New work is **feature-flagged off by default** when it touches AI or external calls.
- **Never edit applied migrations** — fix forward.
- Run the **Track-B audit** at phase boundaries; close high-risk findings before advancing.
- Keep `route:list` collision-free; keep `APP_DEBUG=false` in production.

14. Security-by-design

- Sanctum auth + project policy + module/QynShield entitlement on every tenant route.
- Audit sensitive actions.
- Prompt logging, conversation persistence, RAG, and tenant-evidence embeddings **off by default.**

- Secrets in env/secrets manager; `.env` never committed.

AI Adapter Platform principles (Sprint 22)

- **One way to add module AI.** Modules expose AI through an `AIModuleAdapter` registered with the `AIModuleAdapterRegistry`. There is **no per-module branching** (`if module == ...`) anywhere in the platform; capabilities, actions, and navigation are **discovered** dynamically.
- **One provider-calling point.** All module intelligence narrates through the shared `ModuleAINarrator`. Adapters and services **never** call a provider directly and **never** duplicate provider, prompt, reasoning, retrieval/RAG, or orchestration logic.
- **Backward-compatible contracts.** Contract growth (e.g. new metadata methods) ships with safe defaults (`AbstractAIModuleAdapter`) so existing adapters keep working unchanged.
- **Evidence or honesty.** Adapters ground every answer in **real** domain data; capabilities without a data source are exposed but reported as *not collected* — never fabricated.
- **Same governance for every adapter.** Workspace-scoped, entitlement-gated, RBAC-gated, capability-gated, audited, provider/conversation-logged, rate-limited, and UUID-only.

Automation standards (Sprint 23)

These extend the standards above for any code that executes actions:

- **Registry-driven, never hardcoded.** Execution goes through the `AutomationAdapterRegistry` and `ActionRegistry`. No module embeds its own execution logic; future runners register, they don't fork.
- **Safe by default.** Dry-run is the default. Live, destructive, or side-effecting actions require: the global live switch ON, the runner enabled, the target allowlisted (HTTP), and human approval.
- **Honest results.** Adapters return `dry_run` or `skipped` when they cannot truly run — they never fabricate output, mirroring the platform's "no fake data" rule.
- **Reversible & audited.** Prefer `supportsRollback()`; every execution, approval, and rollback is audited; UUID-only, workspace-isolated, RBAC-gated.
- **Evidence-based AI, no auto-execution.** AI may draft/recommend (citing auditable learning records) but never auto-executes. Learning is inspectable history — **no model training, no hidden state**.

Knowledge & collaboration standards (Sprint 24)

These extend the standards above for knowledge, search, and collaboration code:

- **Registry-driven knowledge.** Knowledge providers go through the `KnowledgeSourceRegistry` implementing the `KnowledgeSource` contract. **No provider-specific branching** in

search/graph/timeline — there is no `if (source == 'confluence')` anywhere; the registry is the only seam.

- **Read-models, not duplicates.** Enterprise Search, Knowledge Graph v2, and the Global Timeline read real rows and write nothing. They never duplicate, cache-mutate, or fabricate events.
- **Real data only.** Search returns only real indexed rows; planned/unwired sources report `isOperational() === false` and are skipped. Notification routing selects only real workspace members.
- **Honest AI.** Knowledge Assistant, Ticket, and Notification Intelligence narrate exclusively through `ModuleAiNarrator`, are grounded in retrieved evidence, are **editable and never auto-applied**, and say "**insufficient evidence**" rather than inventing assignees, owners, or facts.
- **One collaboration layer.** Comments/mentions/participants are polymorphic over `(entity_type, entity_uuid)`; modules reuse the shared `CollaborationService / CollaborationPanel` — never a private copy.

Enterprise Intelligence standards (Sprint 25)

These extend the standards above for event-driven, context, and intelligence code:

- **No module-to-module calls.** Cross-module communication goes through the **Platform Event Bus** (`PlatformEventBus`) using canonical `PlatformEventNames`. Publishers don't know subscribers; subscriber failures are isolated; every publish is audited. **No** `if (module == ...)` branching, ever.
- **One context source.** AI context is built **only** by the `EnterpriseContextEngine` — never hand-assembled in a module. It is a read-model and must stay recursion-safe.
- **Operators recommend, modules execute.** QynVA proposes **editable, evidence-based** plans referencing existing module actions and **never executes**. Cross-module execution happens through the owning module's approved, audited path after human approval.
- **No fabricated values.** Analytics/Executive return honest `available:false`; QynBalance returns counts
 - "pricing unavailable" unless real rates are configured. Never invent financial, capacity, or KPI numbers.
- **Self-observability.** Platform-level state is exposed through Platform Health and Enterprise Analytics as read-models over real signals — never mocked status.

The standard, in one line: *Deterministic, provenance-backed, fail-closed, UUID-first, AI-as-renderer — with no fake data, no fabricated claims, and no unapproved actions, ever.*